作业内容（简答题）

题目一：银行多线程任务调度系统

某银行有一批业务任务需采用多线程架构处理，按以下策略调度：

1. 优先级从高到低：VIP 业务 > 普通业务 > 对公业务
2. 相同优先级内：采用非抢占式短作业优先（SJF）

请编程实现满足业务需求的简单系统，开发语言、框架不限。

题目二：Web 服务器集群负载均衡模拟系统

有一个 Web 服务器集群，通过负载均衡器接收客户端 HTTP 请求，自动将请求均匀分配给空闲服务器处理。若存在多个空闲服务器，则可以设计适当的算法来选择。

请编程来模拟该服务器（用线程来模拟服务器）集群的请求分发过程，开发语言、框架不限。分配算法可自行设计。

选题要求

请学号尾数为单号的同学做第一题，双号的同学做第二题。

作业要求

1. 可以使用任何 AI 编程工具完成编程工作。
2. 提供和 AI 编程工具交互的全过程截图，需在截图中明确体现该题目为本人完成的痕迹。
3. 能解读 AI 生成的代码（核心功能），形成一份代码说明并提交。

4. 作业讲评环节，会选取部分同学上讲台介绍作业完成过程。

选题说明

学号尾数为双数，完成题目二。

作业完成过程记录

以下记录了使用 AI 工具完成本作业的全过程：

步骤 1：需求分析

Web 服务器集群负载均衡模拟系统说明文档

1. 系统概述

本模拟系统通过多线程技术模拟现代 Web 服务器集群的并发处理能力。系统由"模拟服务器"和"负载均衡器"两个核心组件构成，模拟高并发场景下的资源调度与分配。

2. 核心架构设计

组件	类名	职责
模拟服务器	`WebServer`	继承自线程基类，使用 `time.sleep()` 模拟请求处理耗时
负载均衡器	`LoadBalancer`	充当资源管理器，负责对处理机资源实施有效管理与分配

3. 调度算法：时间片轮转（Round Robin）

特性	说明
核心逻辑	将就绪服务器视为循环队列，按顺序指派请求
公平性保证	采用取余运算 `(index + 1) % N` 实现循环指派
算法优势	不要求预知任务时长，更适合 Web 请求场景

4. 同步机制：互斥锁保护

项目	说明
临界区	负载均衡器更新"当前服务器索引"的代码段
保护机制	使用 `threading.Lock()` 互斥量
操作流程	分发请求前执行加锁，确保操作原子性

5. 性能评估准则

指标	说明
响应性能	确保每个请求在最短时间内得到响应
系统吞吐量	利用多线程并行处理，提升单位时间处理总数

作业完成流程记录

步骤 1：需求分析

步骤 2：系统设计与架构

```
Claude Code
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

PS F:\biji\笔记\笔记1\04-学习资料\操作系统> claude
  Claude Code v2.1.87
                                   Welcome back!

                                      ▐▛▜▌
                                     ▝▜▄▛▘

                                 kimi-k2.5 with high effort · API Usage Billing
                                    F:\biji\笔记\笔记1\04-学习资料\操作系统

                                   Tips for getting started
                                   Run /init to create a CLAUDE.md file with instructions for Cla…

                                   Recent activity
                                   No recent activity

❯ 阅读这一个作业的文档"F:\biji\笔记\笔记1\04-学习资料\操作系统\作业.md"然后先完成wEB服务器的模拟过程，然后将代码保存在
  代码文件夹下
```

```
• Read OS assignment and im
    4 基于时间片轮转(Round Robin)算法的多线程服务器集群模拟
    5
    6 作者：[你的名字]
    7 学号：[你的学号]
    8 日期：2026-04-06
    9 """
   … +269 lines (ctrl+o to expand)

● 代码已保存，让我测试运行一下确保正常工作。

● Bash(cd "F:/biji/笔记/笔记1/04-学习资料/操作系统/代码" && python web_server_load_balancer.py)
  └ Exception in thread Server-0:
    Traceback (most recent call last):
      File "C:\Users\admin\AppData\Local\Programs\Python\Python311\Lib\threading.py", line 1045, in _bootstrap_inn
    … +38 lines (ctrl+o to expand)
  └ (timeout 1m)

● 有编码问题，我来修复一下，用ASCII字符替代Unicode符号。

  Reading 1 file… (ctrl+o to expand)
  └ web_server_load_balancer.py

✢ Hatching… (1m 59s · ↓ 2.4k tokens · thinking with high effort)
  └ Tip: Use /btw to ask a quick side question without interrupting Claude's current work

❯

  ▸▸ accept edits on (shift+tab to cycle) · esc to interrupt
```

步骤 3：代码实现过程

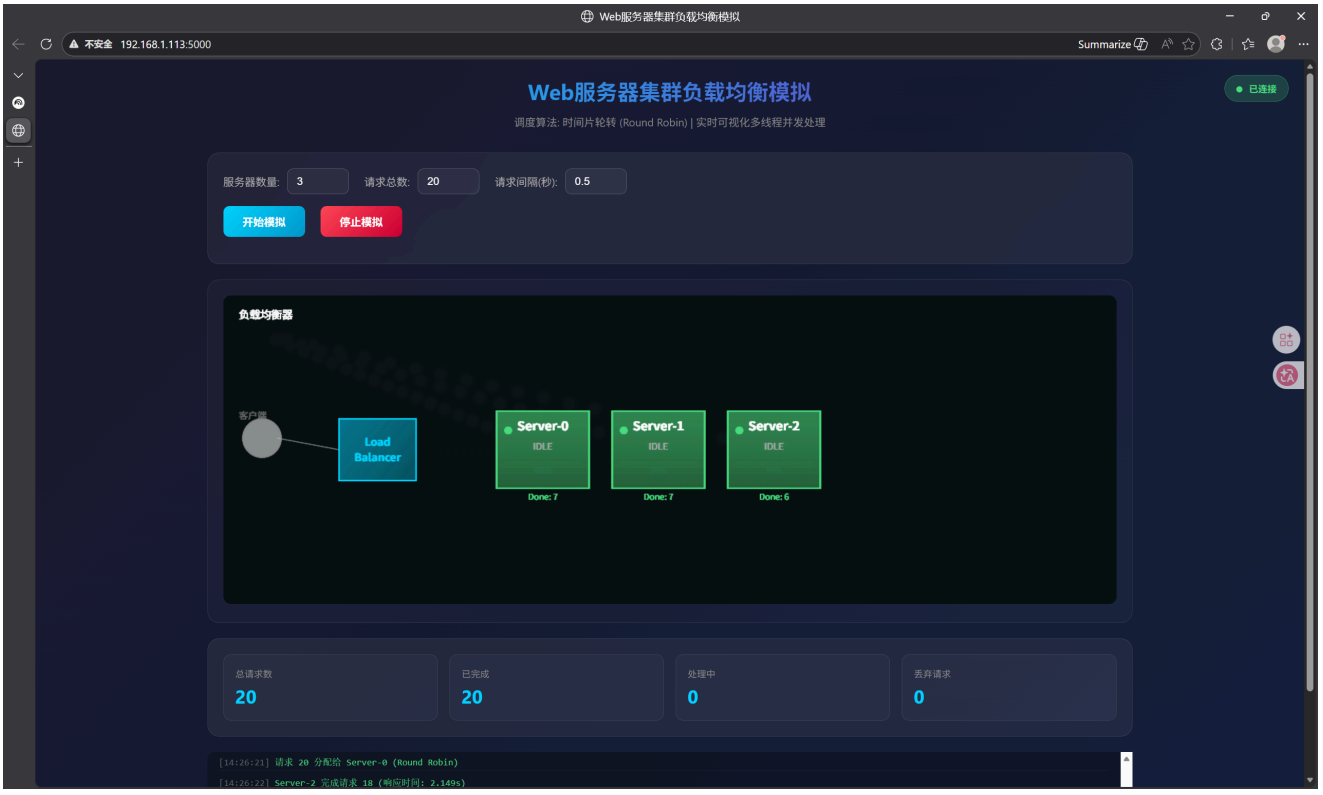
```
Read OS assignment and ii
    169
    170     def get_stats(self) -> dict:

● Bash(cd "F:/biji/笔记/笔记1/04-学习资料/操作系统/代码" && python web_server_load_balancer.py)
  └ ============================================================
    Web◆◆◆
```

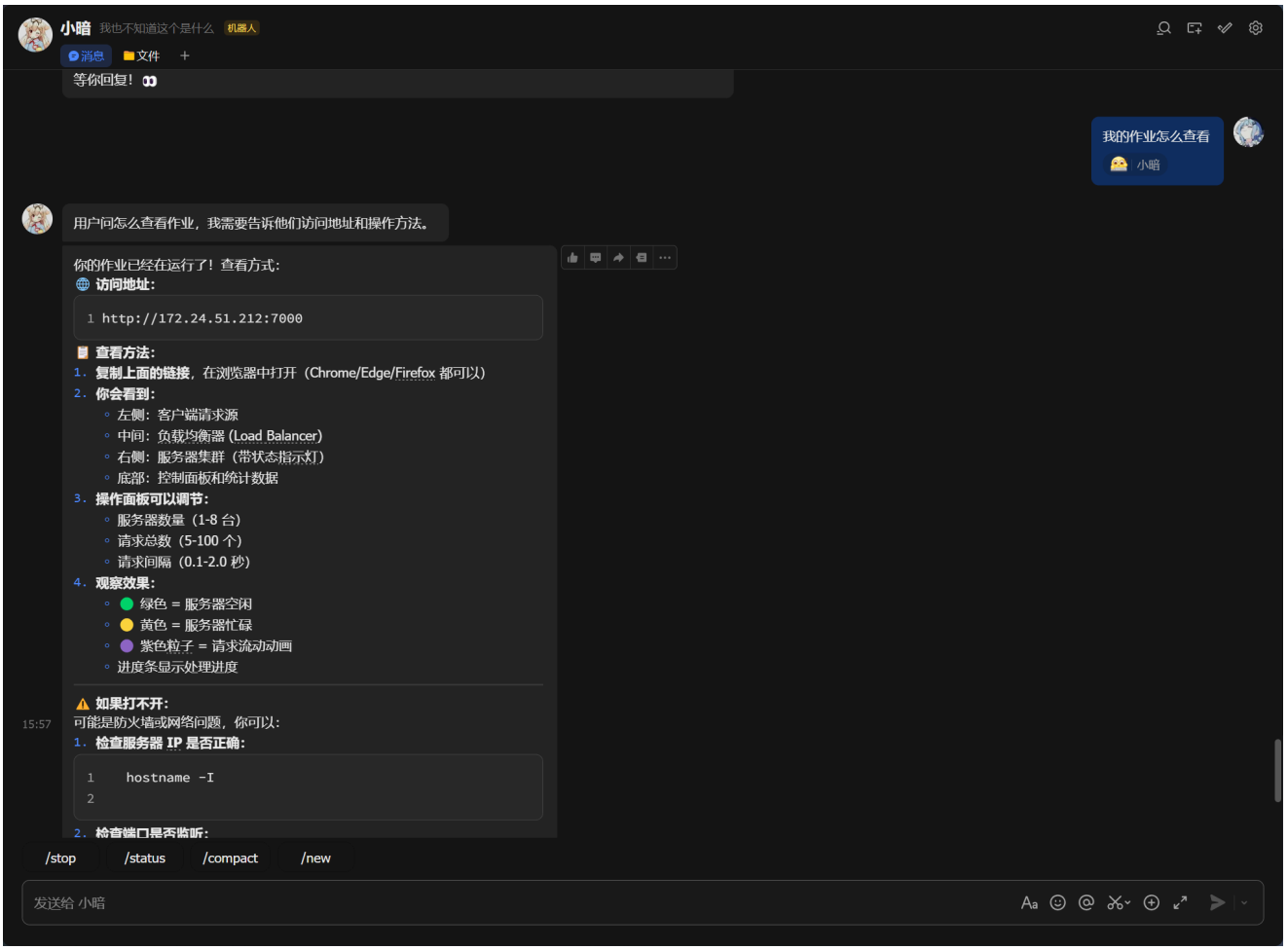
```
● 已完成前后端分离的版本！保存在：

  F:/biji/笔记/笔记1/04-学习资料/操作系统/代码/web_server_simulation/
  ├── server.py          # 后端 - 模拟逻辑
  ├── templates/
  │   └── index.html     # 前端 - Canvas动画
  └── README.md          # 使用说明

  运行方法

  cd "F:/biji/笔记/笔记1/04-学习资料/操作系统/代码/web_server_simulation"

  # 安装依赖
  pip install flask flask-socketio

  # 启动服务
  python server.py

  然后浏览器打开 http://localhost:5000

  前后端分工
```

后端（server.py）	前端（index.html）
多线程服务器模拟	Canvas动画渲染
时间片轮转算法	服务器状态可视化
互斥锁保护临界区	请求流动画（紫色粒子）
WebSocket推送数据	进度条实时更新
记录处理日志	统计面板展示

```
  动画效果

  - 🟢 绿色服务器 = 空闲
  - 🟡 黄色服务器 = 忙碌（带进度条）
  - 🟣 紫色粒子 = 请求流向服务器
  - 📊 实时统计面板

  这样设计既体现了操作系统的多线程、临界区保护等核心概念，又有直观的可视化效果！

✻ Churned for 4m 51s

❯ 帮我检查依赖有没有安装，安装完之后帮我启动服务

  ▸▸ accept edits on (shift+tab to cycle)
```

步骤 6：部署上线与公网测试





将文件上传到服务器，使用 Copow 部署 Python 环境，并通过公网访问进行测试。

代码说明

代码文件位置

代码文件位于 `04-学习资料/操作系统/代码/` 目录下：

```
代码/
├── web_server_load_balancer.py        # 命令行版本（基础多线程实现）
└── web_server_simulation/             # 前后端分离版本（Flask + Canvas动画）
    ├── server.py                      # 后端服务
    ├── templates/index.html           # 前端页面
    └── README.md                      # 运行说明
```

运行方式

命令行版本

```
cd 代码
python web_server_load_balancer.py
```

前后端版本

```
cd 代码/web_server_simulation
pip install flask flask-socketio
python server.py
# 然后在浏览器中打开 http://localhost:7000
```

公网部署版本

```
# 将项目上传到服务器后
python server.py
# 通过服务器公网IP访问: http://<服务器公网IP>:7000
```

核心功能说明

WebServer 类（模拟服务器）

- 继承自 `threading.Thread`，每个服务器作为独立线程运行
- 使用 `queue.Queue` 存储待处理请求
- `time.sleep()` 模拟请求处理耗时
- 提供 `is_busy()` 方法检查服务器状态

LoadBalancer 类（负载均衡器）

- 实现时间片轮转（Round Robin）算法
- 使用 `threading.Lock()` 保护 `current_index` 更新（临界区）
- 优先分配请求给空闲服务器，保证负载均衡

Request 类（HTTP请求）

- 包含请求ID、客户端IP、处理时间等属性
- 记录创建时间、开始时间、完成时间用于统计

技术要点

操作系统概念	实现方式
多线程	每个服务器是 `Thread` 子类
临界区	用 `Lock` 保护 `current_index`
调度算法	时间片轮转 (Round Robin)
进程同步	队列+锁实现安全传递

核心功能解读

系统两大核心设计

1. 多线程并发处理架构

系统采用多线程设计，每个 WebServer 实例都是一个独立的线程：

```
class WebServer(threading.Thread):
    def run(self):
        while self.active:
            request = self.request_queue.get(timeout=0.1)
            self.process_request(request)
```

OS 知识点对应：线程是操作系统调度的基本单位，多个线程并发执行，提高了系统的并发处理能力。

2. 时间片轮转（Round Robin）调度算法

负载均衡器使用 Round Robin 算法将请求均匀分配给服务器：

```
self.current_index = (self.current_index + 1) % self.num_servers
```

OS 知识点对应：Round Robin 是操作系统中经典的调度算法，通过时间片轮转实现公平调度。

3. 互斥锁保护临界区

当多个请求同时竞争 `current_index` 时，使用互斥锁保证数据一致性：

```
with self.lock:
    self.current_index = (self.current_index + 1) %
self.num_servers
```

OS 知识点对应：临界区保护是并发编程的基础，通过 Lock 实现互斥访问，防止竞态条件。

核心流程图

```
┌──────────────┐
│  客户端请求  │
└──────────────┘
       │
       ▼
┌──────────────────────┐
│  负载均衡器          │
│  (LoadBalancer)      │
│  Round Robin算法     │
└──────────────────────┘
       │
```

```
        ├──────────┬──────────┬──────────┤
        ▼          ▼          ▼          ▼
┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐
│Server-1│  │Server-2│  │Server-3│  │Server-N│
│(线程)  │  │(线程)  │  │(线程)  │  │(线程)  │
└─────────┘  └─────────┘  └─────────┘  └─────────┘
```

与课堂知识的联系

课堂知识点	本程序体现
进程/线程调度	服务器线程被 CPU 调度执行
时间片轮转(RR)	Round Robin 算法分配请求
多线程并发	多个服务器线程并行处理
临界区保护	Lock 保护 shared variable
生产者-消费者	请求生成与处理的解耦